```
"""Live user settings — shared across all gameplay modules.

scanlister.py owns the authoritative variables (loaded from settings_store) and
pushes them into this module via push_from_raw() whenever a setting changes.
Gameplay code reads SETTINGS.* so we don't have to thread 30 parameters
through every function.

Keep this module dependency-free (no pygame, no state) so it imports cleanly
from anywhere.
"""

COLORBLIND_MODES = ("OFF", "protanopia", "deuteranopia", "tritanopia")

class _Settings:
    def __init__(self):
        # -- Display --
        self.fullscreen = False
        self.brightness = 1.0          # 0.5 .. 1.5 multiplier (50 -> 1.0 neutral)
        self.scanlines = True
        self.film_grain = True
        self.reduced_motion = False
        self.colorblind_mode = "OFF"

        # -- Audio --
        self.master_volume = 1.0       # 0..1
        self.music_volume = 0.7
        self.sfx_volume = 0.85
        self.subtitles = True
        self.mute_on_focus_loss = True

        # -- Gameplay --
        self.mouse_sensitivity = 1.0   # 0.25 .. 3.0
        self.invert_mouse_y = False
        self.aim_assist = False
        self.auto_reload = True
        self.particle_density = 1.0    # 0.0 .. 1.5

        # -- HUD --
        self.show_crosshair = True
        self.crosshair_size = 6        # scaled baseline bb-large (pixel half-extent)

    def push_from_raw(self, raw: dict):
        """Update from the raw settings_store dict (0-100 ints, bits for modes).

        This is the single conversion point: scanlister.py calls this every time
        any setting changes; gameplay reads the already-converted float values.
        """
        if "display_mode" in raw:
            self.fullscreen = bool(raw["display_mode"])
        if "brightness" in raw:
            # 0..100 with 50 = neutral; map to 0.5 .. 1.5
            self.brightness = 0.5 + (raw["brightness"] / 100.0)
        if "scanlines" in raw:
            self.scanlines = bool(raw["scanlines"])
        if "grain" in raw:
            self.film_grain = bool(raw["grain"])
        if "reduced_motion" in raw:
            self.reduced_motion = bool(raw["reduced_motion"])
        if "colorblind_mode" in raw:
            idx = int(raw["colorblind_mode"])
            if 0 <= idx < len(COLORBLIND_MODES):
                self.colorblind_mode = COLORBLIND_MODES[idx]

        if "master_volume" in raw:
            self.master_volume = raw["master_volume"] / 100.0
        if "music_volume" in raw:
            self.music_volume = raw["music_volume"] / 100.0
        if "sfx_volume" in raw:
            self.sfx_volume = raw["sfx_volume"] / 100.0
        if "subtitles" in raw:
            self.subtitles = bool(raw["subtitles"])
        if "mute_on_focus_loss" in raw:
            self.mute_on_focus_loss = bool(raw["mute_on_focus_loss"])

        if "mouse_sensitivity" in raw:
            # 0..100 with 50 = 1.0; map to 0.25 .. 3.0 (roughly linear)
            self.mouse_sensitivity = max(0.25, raw["mouse_sensitivity"] / 50.0)
        if "invert_mouse" in raw:
            self.invert_mouse_y = bool(raw["invert_mouse"])
        if "aim_assist" in raw:
            self.aim_assist = bool(raw["aim_assist"])
        if "auto_reload" in raw:
            self.auto_reload = bool(raw["auto_reload"])
        if "particle_density" in raw:
            # 0..100 -> 0.0 .. 1.5 (100 = 1.5x denser than baseline)
            self.particle_density = (raw["particle_density"] / 100.0) * 1.5

        if "crosshair" in raw:
            self.show_crosshair = bool(raw["crosshair"])
        if "crosshair_size" in raw:
            self.crosshair_size = int(raw["crosshair_size"])

SETTINGS = _Settings()
```